Deadlock Prevention in a Blocking Two-Machine Manufacturing Cell with a Single Petri-Net Control Place:

Proofs, Closed-Form Results, and a Cross-Validated C++ Execution Engine

Mohamed Ghoneim
King Fahd University of Petroleum and Minerals
g202522450@kfupm.edu.sa

October 2026

Abstract

In blocking flow lines, a finished part keeps its machine until the next machine accepts it; when two products traverse the same machines in opposite order, a circular wait can freeze the cell. We study the smallest instance that exhibits this failure—two single-capacity machines, products with routings $M_1 \to M_2$ and $M_2 \to M_1$, no buffers—and ask: *how much production concurrency can be retained while preventing deadlock?* We model the cell as a place/transition Petri net in which completing an operation never releases a machine. For a $2A{+}2B$ batch, exhaustive analysis finds 97 reachable markings, 4 terminal deadlocks, and 16 *doomed* markings from which the batch can no longer finish—so the point of no return precedes the visible stall. A single control (monitor) place, a generalized mutual-exclusion constraint on first-stage occupancy, removes every doomed marking. We prove, for every batch size, that this supervisor is deadlock-free, nonblocking, maximally permissive, and *transparent* (it never alters a run that unrestricted dispatch would have completed), and we derive closed forms for state-space size (e.g. $17n_An_B{+}5n_A{+}5n_B{-}7$ markings), makespan, and the saving over a conservative "one active part" policy. In timed experiments the supervisor shortens the makespan by 10–25% (mean 17.8%) relative to that policy and attains the exact optimum in 162 of 162 scenarios; a Monte Carlo study over 16 000 staggered-arrival scenarios shows 0 deadlocks in 32 000 controlled runs, against 20–100% deadlock probability for unrestricted dispatch. A Python reference and a C++17 execution engine read the same net file and agree event-for-event on 900 timed simulations and 1 201 replayed sequences (23 465 steps). The engine fires a transition in about 24 ns and runs a complete $2A{+}2B$ batch in 0.49 μs using 3.4 MB of resident memory on a laptop CPU. Monitor places for mutual-exclusion constraints are an established technique; our contributions are the cell-specific proofs and closed forms, the timed evaluation, and a validated, fully reproducible implementation.

Keywords: Petri nets; deadlock prevention; supervisory control; blocking flow lines; manufacturing systems; execution engine; reproducibility.

1 Introduction

Automated manufacturing cells frequently pass parts directly from one machine to the next without intermediate storage. In such *blocking* lines, a finished part keeps occupying its machine until the next machine can accept it. If two product flows visit the same machines in opposite order, the cell can reach a *circular wait*: a part that has finished on machine M_1

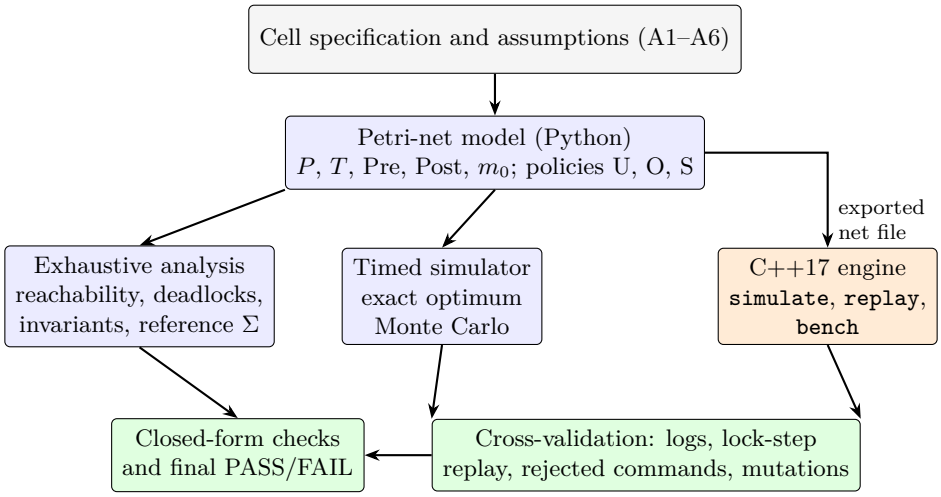


Figure 1: Workflow of this study. One Petri-net model drives exhaustive analysis, timed simulation and an exact-optimum search in Python; the same net, exported as a text file, drives an independent C++17 engine. Cross-validation compares the two implementations, and a final script re-checks every headline claim.

waits for M_2, which is held by a part that has finished on M_2 and waits for M_1. No machine event can resolve this situation, and the cell stops with unfinished parts [1, 6, 12].

Deadlock control for automated manufacturing systems is a mature field with several surveys [12, 25, 26]. Methods are usually classified as prevention, avoidance, and detection with recovery, and Petri nets are the dominant formalism: they expose the resource structure, support invariant and siphon analysis, and yield supervisors that are themselves nets [11, 17, 24]. The general problem is hard—maximally permissive deadlock avoidance is NP-hard for broad classes of resource allocation systems [23, 35]—which is why most of the literature develops sufficient policies that trade permissiveness for computational cost [25]. Very small instances, in contrast, can be analysed *completely*. That permits statements that are proved rather than merely demonstrated, and software that can be checked against an independent implementation.

This report. We study the smallest cell that exhibits the circular wait (Section 3) and ask a single question:

> *How much production concurrency can be retained while preventing resource deadlocks?*

We answer it on three levels: *logical* (which behaviours can a deadlock-free supervisor possibly allow?), *timed* (what does that permissiveness buy in makespan, and how far is it from the optimum?), and *computational* (what does it cost to execute the supervisor, and can the execution be trusted?). Figure 1 summarises the workflow.

Contributions.

1. **Model and structure.** A Petri-net model in which completing an operation never releases a machine (Section 3); its place invariants; and a progress function showing that every complete run has exactly $5(n_A + n_B)$ firings (Section 5).

2. **Exact characterisation of the failure.** The terminal deadlocks are exactly $n_A n_B$

markings; the set of *doomed* markings is an absorbing region with $4n_An_B$ members; the shortest route to deadlock has four firings (Theorem 4 and Proposition 5).

3. **A provably adequate supervisor.** One monitor place enforces $\varphi \leq 1$ (at most one part in a first-stage position). For every batch size it is deadlock-free, nonblocking (Theorem 6), maximally permissive (Theorem 8) and transparent (Proposition 10).

4. **Closed forms.** State-space sizes, deadlock and doomed counts, the number of states with both machines processing, makespans, and the saving over the conservative policy (Propositions 11 and 12); all machine-checked against exhaustive enumeration and simulation.

5. **Timed evaluation.** A deterministic grid of 162 scenarios and a Monte Carlo study of 16 000 scenarios with common random numbers (Section 7); an exact timed search shows the supervisor reaches the optimum makespan.

6. **A validated, reproducible engine.** A C++17 execution engine and a Python reference that share one net file; a cross-validation protocol (differential testing, lock-step replay, rejected commands, mutation tests); resource measurements; and a single command that regenerates every result (Section 6).

Positioning and honesty. The supervisor is a *monitor place* enforcing a generalized mutual-exclusion constraint (GMEC) in the sense of Giua, DiCesare and Silva [15] and Yamalidou et al. [41]; that monitors enforcing an admissible GMEC are maximally permissive with respect to that constraint is known [15, 30]. We therefore do *not* claim a new deadlock-prevention algorithm. What this report adds is a complete, proved treatment of one cell in which this particular constraint coincides with the supremal *nonblocking* behaviour for every batch size, the timed and computational evaluation, and an implementation whose agreement with an independent one is demonstrated rather than assumed. We do not model machine failures, non-atomic transfers, buffers, or larger layouts, and nothing here has been run on a PLC or real machine; Section 9.3 states the limits explicitly.

Organisation. Section 2 reviews related work. Sections 3 and 4 define the cell, the net and the three control policies. Section 5 contains the theory. Section 6 describes the engine and its validation, Section 7 the experiments, Section 8 the results, and Section 9 the discussion and limitations.

2 Related work

Deadlock in resource allocation. Coffman et al. [6] identified the conditions under which resource deadlock arises: mutual exclusion, hold-and-wait, no preemption, and circular wait. In blocking manufacturing lines, hold-and-wait is structural: a part that has finished processing still holds its machine. For flexible manufacturing systems (FMS), Banaszak and Krogh [1] gave a Petri-net model of concurrently competing process flows and a restriction policy that avoids deadlock, and Viswanadham et al. [37] compared prevention by reachability analysis with a net-based on-line avoidance controller. Safety of a resource-allocation state—whether some sequence of allocations completes all processes—is NP-complete for broad classes of sequential resource allocation systems, and maximally permissive avoidance is correspondingly hard [23, 35]. Scheduling theory studies the same blocking behaviour from the makespan side; for two machines, the blocking flow shop has a simple closed form for identical jobs [16], which we use in Proposition 12.

Petri nets for modelling and analysis. Place/transition nets and their analysis methods—reachability, coverability, structural invariants—are surveyed in [7, 31, 34]. Minimal-support place invariants can be computed with the Farkas-type algorithm of Martínez and Silva [27], which we implement and use to obtain the conservation laws of Lemma 2. Established tools include TINA, which builds state-space abstractions and computes semiflows [3], and CPN Tools for coloured nets [20]. Our analysis code is deliberately small and special-purpose; it has not been compared against these tools, and we list that as future work (Section 9.3).

Supervisory control and Petri-net supervisors. In the Ramadge–Wonham framework [4, 32, 40], a supervisor restricts controllable events so that a specification holds, cannot disable uncontrollable events, and should be *nonblocking*; the supremal controllable and nonblocking behaviour is the reference for permissiveness. Our set Σ in Theorem 8 is the state-based analogue for this cell. For Petri nets, Holloway et al. [17] survey control methods; Giua et al. [15] introduced generalized mutual-exclusion constraints (GMECs) $L\,m \le b$ on nets with uncontrollable transitions and showed how to enforce them with *monitor places*; Yamalidou et al. [41] derived such controllers from place invariants; and Moody and Antsaklis [30] and Iordache and Antsaklis [19] develop the theory, including uncontrollable and unobservable transitions. Giua and DiCesare [14] study blocking and controllability of Petri nets under supervision.

Deadlock prevention for manufacturing systems. Ezpeleta et al. [11] characterise deadlocks in a class of nets (S^3PR) of sequential processes sharing resources as empty siphons and prevent them by adding control places. Surveys [12, 25, 26] compare policies by structural complexity, behavioural permissiveness and computational cost, and Li and Zhou [24] treat monitor implementation of GMECs and elementary-siphon-based control in depth. The cell studied here is of the kind such methods address; we derive its monitor directly from the structure of the circular wait and do not use siphon analysis.

Timed Petri nets. Deterministic firing delays were introduced by Ramchandani [33]; Merlin and Farber [29] proposed time Petri nets with firing intervals, whose state-class analysis is due to Berthomieu and Diaz [2]. We use deterministic enabling-time semantics for machine completions; interval semantics, which would model processing-time uncertainty, is future work.

From nets to controllers. Programmable controllers are programmed in languages such as those of IEC 61131-3 [18], and IEC 61499 function blocks offer a distributed alternative [38]. Kaid et al. [21] convert a Petri-net deadlock-control model into ladder diagrams and test the result on a laboratory manufacturing system. Our engine instead *interprets* the net file directly, so the model analysed and the model executed are the same artifact; whether this is preferable to code generation depends on the target and is outside our scope. A conceptual parallel exists in continuous control: reference governors enforce constraints by modifying commands [13], and cascaded reference-governor/MPC schemes have been applied to two-stage manufacturing machines [9]. A control place likewise restricts the controllable commands that would leave the admissible set. This is an analogy, not an equivalence.

Validating implementations. We rely on differential testing [28] between two implementations of the same semantics, on mutation analysis [8] to show that our checks are not vacuous, and on randomised property checks in the spirit of [5]. Reproducibility practices follow [36]; the statistics are standard [10, 22, 39].

Table 1: Positioning of this report against established results.

Established	**Added here (for the cell of Section 3)**
Monitor places enforce admissible GMECs maximally permissively [15, 30]	The specific constraint $\varphi \leq 1$ equals the supremal *non-blocking* behaviour for every batch size (Theorem 8); transparency (Proposition 10)
General deadlock-prevention policies for S^3PR and AMS; permissiveness comparisons [11, 25]	Exact closed forms for a minimal cell: state counts, $n_A n_B$ deadlocks, $4n_A n_B$ doomed markings (Proposition 11)
Timed Petri nets; blocking flow shops [16, 33]	Closed-form makespans of the supervised cell and the saving over a conservative policy; exact optimum (Proposition 12)
Petri-net supervisors compiled to PLC code [21]	A net-interpreting engine validated against an independent implementation; measured resources; no hardware claim

3 Problem and model

3.1 The cell and its assumptions

The cell consists of two machines M_1, M_2 and two products. Product A is processed first on M_1 and then on M_2; product B first on M_2 and then on M_1. We assume:

(A1) each machine processes at most one part at a time and holds at most one part;

(A2) there are no buffers: a part that has finished processing stays on its machine until it is moved on;

(A3) a batch consists of n_A parts of A and n_B parts of B; parts of one product are interchangeable and served first-in first-out (tokens are not individually identified);

(A4) machine completions are observable and *uncontrollable*, with positive integer delays (*ticks*); loading, transferring and unloading are *controllable* commands executed instantaneously and atomically—a transfer frees the first machine and occupies the second in one step, and requires the second to be free;

(A5) machines are reliable and the output always accepts finished parts;

(A6) parts are available at $t = 0$ or arrive at known external times.

Let α_1, α_2 denote the processing times of A on M_1 and M_2 (stage 1 and stage 2), and β_1, β_2 those of B on M_2 (stage 1) and M_1 (stage 2). The circular wait is immediate: A finished on M_1 needs M_2, while B finished on M_2 needs M_1.

3.2 Place/transition nets

Definition 1 (Net)**.** A *net* is $N = (P, T, \mathrm{Pre}, \mathrm{Post}, m_0)$ with finite sets of places P and transitions T, arc-weight matrices $\mathrm{Pre}, \mathrm{Post} \in \mathbb{N}^{P \times T}$, and initial marking $m_0 \in \mathbb{N}^P$. A transition t is *enabled* at m if $m \geq \mathrm{Pre}(\cdot, t)$; firing it yields $m' = m + C(\cdot, t)$ with incidence matrix $C = \mathrm{Post} - \mathrm{Pre}$. $\mathcal{R}(N)$ is the set of markings reachable from m_0. A marking is *terminal* if no transition is enabled. $T = T_\mathrm{c} \dot{\cup} T_\mathrm{u}$ splits into controllable and uncontrollable transitions. A vector $y \in \mathbb{N}^P$ with $y^\top C = 0$ is a *P-invariant*: $y^\top m = y^\top m_0$ for all $m \in \mathcal{R}(N)$.

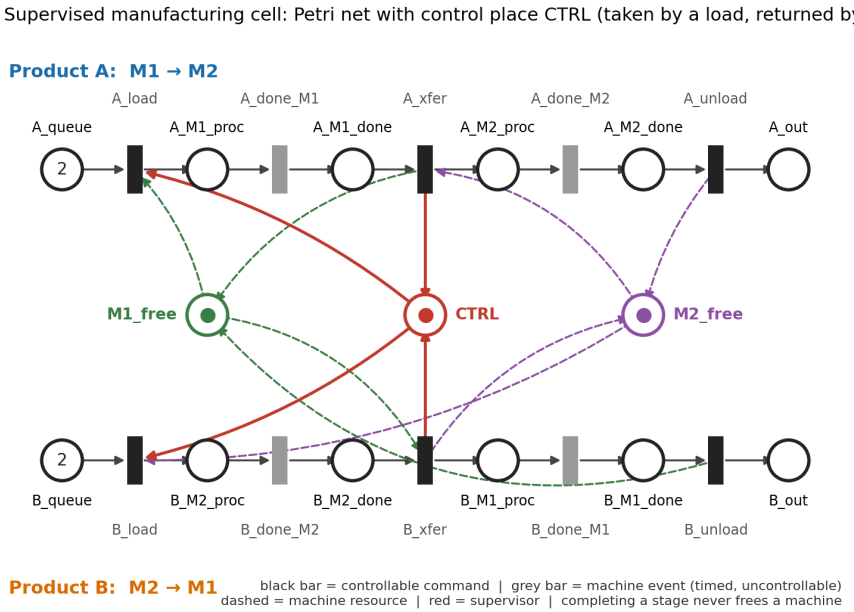


Figure 2: The supervised cell. Product A (top row) visits M_1 then M_2; product B (bottom row) visits M_2 then M_1. Black bars are controllable commands, grey bars uncontrollable timed machine events. Dashed arcs are the machine resources F_1 (green) and F_2 (purple); the red control place C is taken by the two loads and returned by the two transfers. Removing C gives the unrestricted plant.

3.3 The plant net

The uncontrolled cell is the net $N_U(n_A, n_B)$ of Figure 2 and Table 2 (with the control place C of the figure removed). It has 14 places and 10 transitions. For stage $s \in \{1, 2\}$, places A_s^{p} and A_s^{d} hold an A part that is *processing* or *done* (finished, still holding its machine) in stage s, and analogously $B_s^{\mathrm{p}}, B_s^{\mathrm{d}}$ for B; Q_A, Q_B are the queues, O_A, O_B the outputs, and F_1, F_2 the availability tokens of the machines. The initial marking is $Q_A = n_A$, $Q_B = n_B$, $F_1 = F_2 = 1$. The *final marking* m_{f} has $O_A = n_A$, $O_B = n_B$, both machines free, and nothing else marked.

The key modelling rule is that a completion event $(e_{A1}, e_{A2}, e_{B1}, e_{B2})$ only moves a token from a *processing* to a *done* place and never touches F_1 or F_2; machines are released only by a transfer or an unload command.

3.4 Timed semantics

Time is measured in integer ticks. A transition with delay $\delta > 0$ is *timed*: it has a single timer that starts when the transition becomes enabled and fires δ ticks later (single-server, enabling memory). A transition with delay 0 is *immediate* and is fired by a dispatcher. At every event time t the following steps are executed in this order: (1) external arrivals due at t are injected; (2) timed transitions whose timers expire at t fire, in net order; (3) the dispatcher repeatedly fires the *first enabled immediate transition in net order* until none is enabled; (4) timers of newly enabled timed transitions are started and timers of disabled ones are cleared. The run ends when no timer and no arrival is pending; it is *done* if the marking is final and a *deadlock* otherwise. Net order is fixed and explicit: unloads, then transfers, then loads (with ℓ_A before ℓ_B or vice versa; both orders are evaluated). All times are integers, so runs are exactly reproducible and logs can be compared byte for byte.

Table 2: Transitions of the plant net (T_c: controllable command, T_u: uncontrollable timed event). "Code name" is the identifier in the net file and software.

Symbol	Code name	Type	Input places	Output places	Delay
ℓ_A	`A_load`	T_c	$Q_A,\ F_1$	A_1^p	0
e_{A1}	`A_done_M1`	T_u	A_1^p	A_1^d	α_1
x_A	`A_xfer`	T_c	$A_1^d,\ F_2$	$A_2^p,\ F_1$	0
e_{A2}	`A_done_M2`	T_u	A_2^p	A_2^d	α_2
u_A	`A_unload`	T_c	A_2^d	$O_A,\ F_2$	0
ℓ_B	`B_load`	T_c	$Q_B,\ F_2$	B_1^p	0
e_{B1}	`B_done_M2`	T_u	B_1^p	B_1^d	β_1
x_B	`B_xfer`	T_c	$B_1^d,\ F_1$	$B_2^p,\ F_2$	0
e_{B2}	`B_done_M1`	T_u	B_2^p	B_2^d	β_2
u_B	`B_unload`	T_c	B_2^d	$O_B,\ F_1$	0

4 Control policies

We compare three policies, all obtained from the plant by adding at most one place.

U: unrestricted dispatch. No control place; every enabled command may fire.

O: one active part (conservative). A place C with one token is consumed by both loads and returned by both unloads, so at most one part is in the cell.

S: control-place supervisor. A place C with one token is consumed by both loads ℓ_A, ℓ_B and returned by both *transfers* x_A, x_B (Figure 2).

GMEC view. Let $\varphi(m) = m(A_1^p) + m(A_1^d) + m(B_1^p) + m(B_1^d)$ be the *first-stage occupancy*: the number of parts that are processing or finished-and-waiting in stage 1. Policy S enforces the GMEC $\varphi(m) \le 1$. With L the indicator vector of the four stage-1 places, the standard monitor construction [30, 41] gives the control place incidence $D_C = -L\,C$ and initial marking $1 - L\,m_0 = 1$. Since $L\,C(\cdot, t) = 0$ for every $t \in T_u$ (completions move a token between two places of the same group), the constraint is *admissible*: the monitor has no arcs to uncontrollable transitions. Likewise, policy O enforces $\sum_{\text{machine-holding places}} m \le 1$.

Why this constraint. Figure 3 shows the intuition. Unrestricted dispatch can raise φ from 1 to 2 by loading the *other* product while the first is still in stage 1; once $\varphi = 2$, both machines are held by stage-1 parts that need each other's machine, and nothing can lower φ again. The monitor blocks exactly those loads. Returning the token at the *transfer* (rather than at the stage-1 completion) matters: a finished part still holds its machine, so freeing the permission earlier admits the circular wait (a mutation we test in Section 6).

5 Analytical results

Throughout, $x = n_A$ and $y = n_B$ denote the numbers of parts, $N = n_A + n_B$, and N_U, N_O, N_S are the nets of policies U, O, S (with initial marking $C = 1$ for the latter two). All statements

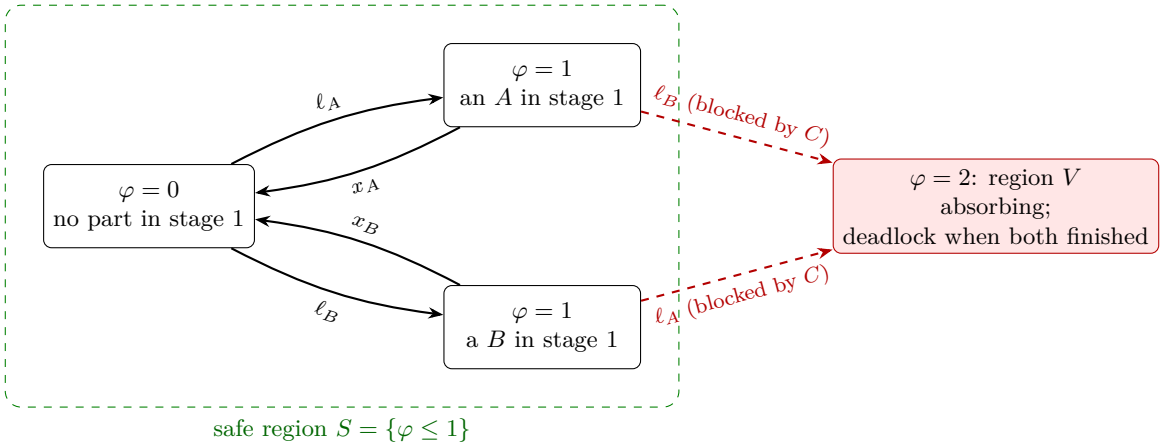


Figure 3: Level graph of first-stage occupancy φ (plant, abstracted). Solid arrows are commands that keep the cell in the safe region $\{\varphi \leq 1\}$; red dashed arrows are the loads that would enter the absorbing region $V = \{\varphi = 2\}$; the control place disables exactly these. Inside V only completion events can fire, and the terminal deadlocks are the markings of V in which both stage-1 parts are finished.

hold for every $n_A, n_B \geq 1$. Computational confirmation is stated explicitly where a proof is not given.

Lemma 2 (Invariants)**.** *The following are P-invariants of N_U (and hence of N_O, N_S, which extend it):*

$$F_1 + A_1^{\mathrm{p}} + A_1^{\mathrm{d}} + B_2^{\mathrm{p}} + B_2^{\mathrm{d}} = 1, \tag{I1}$$

$$F_2 + A_2^{\mathrm{p}} + A_2^{\mathrm{d}} + B_1^{\mathrm{p}} + B_1^{\mathrm{d}} = 1, \tag{I2}$$

$$Q_A + A_1^{\mathrm{p}} + A_1^{\mathrm{d}} + A_2^{\mathrm{p}} + A_2^{\mathrm{d}} + O_A = n_A, \tag{I3}$$

$$Q_B + B_1^{\mathrm{p}} + B_1^{\mathrm{d}} + B_2^{\mathrm{p}} + B_2^{\mathrm{d}} + O_B = n_B. \tag{I4}$$

In N_S additionally $C + A_1^{\mathrm{p}} + A_1^{\mathrm{d}} + B_1^{\mathrm{p}} + B_1^{\mathrm{d}} = 1$ (I5), i.e. $C + \varphi = 1$; in N_O additionally $C + \sum_{\text{holding places}} = 1$ (I5′).

Proof. Each transition removes a token from a place of an invariant and adds one to another place of the same invariant, or does not touch it; this is checked column by column from Table 2. For (I5), ℓ_A, ℓ_B move a token from C to the stage-1 group, x_A, x_B move it back, and the completions e_{A1}, e_{B1} stay inside the group. The Farkas-type algorithm [27] implemented in our code returns exactly these minimal invariants (Section B). □

Consequences: both machines hold at most one part (I1, I2); $\varphi \leq 2$ on $\mathcal{R}(N_U)$; and the final marking is the unique marking with $O_A = n_A$, $O_B = n_B$.

Lemma 3 (Progress)**.** *Assign weights $0, 1, 2, 3, 4, 5$ to the places of each product in the order queue, stage-1 processing, stage-1 done, stage-2 processing, stage-2 done, output, and weight 0 to F_1, F_2, C. Every transition of N_U, N_O, N_S increases the weighted token count Φ by exactly 1. Hence every firing sequence from m_0 has at most $5N$ firings, and every sequence from m_0 to the final marking has exactly $5N$ firings.*

Proof. Each transition moves one part one position along its product's path and touches no weighted place otherwise. $\Phi(m_0) = 0$ and $\Phi(m_{\mathrm{f}}) = 5N$. □

Lemma 3 implies that every maximal run terminates, and it provides a built-in consistency check for the software: complete runs of the $2A{+}2B$ and $100A{+}100B$ batches must have exactly 20 and 1000 firings, which the benchmarks report (Table 9).

Theorem 4 (Deadlocks of the plant)**.** *A marking $m \in \mathcal{R}(N_U)$ is terminal and different from m_{f} if and only if $m(A_1^{\mathrm{d}}) = m(B_1^{\mathrm{d}}) = 1$, all other machine-holding places are empty, and the remaining $n_A - 1$ parts of A and $n_B - 1$ parts of B are distributed arbitrarily between their queues and outputs. There are exactly $n_A n_B$ such markings, and the shortest firing sequence reaching one has length 4.*

Proof. ($\Rightarrow$) Let m be terminal and $m \neq m_{\mathrm{f}}$. Every processing place is empty (otherwise its completion is enabled), and $A_2^{\mathrm{d}}, B_2^{\mathrm{d}}$ are empty (otherwise the unload is enabled). By (I1) and (I2), $F_1 + A_1^{\mathrm{d}} = 1$ and $F_2 + B_1^{\mathrm{d}} = 1$. If $A_1^{\mathrm{d}} = 1$ and $B_1^{\mathrm{d}} = 0$ then $F_2 = 1$ and x_A is enabled; if $A_1^{\mathrm{d}} = 0$ and $B_1^{\mathrm{d}} = 1$ then $F_1 = 1$ and x_B is enabled; if both are 0, both machines are free, so any queued part could be loaded, and with empty queues (I3, I4) give $m = m_{\mathrm{f}}$. Hence $A_1^{\mathrm{d}} = B_1^{\mathrm{d}} = 1$. By (I3), (I4) the other parts are in queues or outputs; with $F_1 = F_2 = 0$ no load is enabled, so every such split is terminal. ($\Leftarrow$) Given a split with i queued and $n_A - 1 - i$ finished A parts and j queued and $n_B - 1 - j$ finished B parts, run the finished parts one at a time through the empty cell, then fire $\ell_A, \ell_B, e_{A1}, e_{B1}$. The splits number $n_A \cdot n_B$. For the length, $\Phi \geq 2 + 2 = 4$ at any such marking (Lemma 3), with equality when all other parts are queued. □

Proposition 5 (Absorbing violation region)**.** *Let $V = \{m \in \mathcal{R}(N_U) : \varphi(m) = 2\}$, i.e. both a stage-1 A part and a stage-1 B part are present. Then (a) no transition leads out of V; (b) V contains every terminal non-final marking; (c) $m_{\mathrm{f}} \notin V$, so every marking of V is* doomed *(the batch can never complete); (d) $|V| = 4n_A n_B$, so $3n_A n_B$ doomed markings are not yet terminal.*

Proof. (a) At $m \in V$, (I1) and (I2) give $F_1 = F_2 = 0$ and $B_2^{\mathrm{p}} = B_2^{\mathrm{d}} = A_2^{\mathrm{p}} = A_2^{\mathrm{d}} = 0$. Then ℓ_A, x_B (need F_1), ℓ_B, x_A (need F_2) and both unloads are disabled; only e_{A1} and e_{B1} can fire, and they leave $\varphi = 2$. (b) by Theorem 4. (c) $\varphi(m_{\mathrm{f}}) = 0$ and V is closed. (d) A marking of V is fixed by: the stage-1 A part is processing or done (2 choices), likewise the B part (2 choices), and the splits of the remaining parts ($n_A \cdot n_B$ choices); all are reachable as in Theorem 4. Of the $4n_A n_B$ markings, $n_A n_B$ are terminal. □

In particular, for $2A{+}2B$ there are 4 terminal deadlocks but 16 doomed markings: the point of no return (the load that raises φ to 2) precedes the visible stall by at least one firing—a completion event—and in general by the remaining processing time of the stage-1 parts, which is why monitoring for a stalled cell reacts too late.

Theorem 6 (The supervisor is deadlock-free and nonblocking)**.** *In N_S: (i) $\varphi \leq 1$ on $\mathcal{R}(N_S)$; (ii) every terminal reachable marking is m_{f} (with $C = 1$); (iii) from every reachable marking, every maximal firing sequence ends in m_{f}, and from m_0 every complete run has exactly $5N$ firings.*

Proof. (i) is (I5) since $C \geq 0$. (ii) Let m be terminal and reachable. As in Theorem 4, all processing places and $A_2^{\mathrm{d}}, B_2^{\mathrm{d}}$ are empty, so (I1), (I2) give $F_1 + A_1^{\mathrm{d}} = 1$, $F_2 + B_1^{\mathrm{d}} = 1$, and (I5) gives $C + A_1^{\mathrm{d}} + B_1^{\mathrm{d}} = 1$. If $C = 0$, exactly one of $A_1^{\mathrm{d}}, B_1^{\mathrm{d}}$ is 1; say $A_1^{\mathrm{d}} = 1$, then $F_2 = 1$ and x_A (which has no precondition on C) is enabled—contradiction; symmetrically for $B_1^{\mathrm{d}} = 1$. If $C = 1$ then $A_1^{\mathrm{d}} = B_1^{\mathrm{d}} = 0$ and both machines are free; a queued part could load, so the queues are empty and $m = m_{\mathrm{f}}$. (iii) By Lemma 3 every sequence is finite, so every maximal one ends in a terminal marking, which is m_{f} by (ii). □

Definition 7 (Safe behaviour)**.** Let $\Sigma \subseteq \mathcal{R}(N_U)$ be the *largest* set such that (a) $m \in \Sigma$, $t \in T_{\mathrm{u}}$ enabled at $m \Rightarrow m + C(\cdot, t) \in \Sigma$, and (b) from every $m \in \Sigma$ the final marking is reachable by a path inside Σ. (It exists: a union of sets with (a), (b) has (a), (b).)

Any supervisor that observes the marking, cannot disable T_{u}, and keeps the closed loop nonblocking has a reachable closed-loop set with properties (a) and (b), hence contained in Σ: Σ bounds the permissiveness of *every* such supervisor.

Theorem 8 (Maximal permissiveness)**.** *(i)* $\Sigma = \{m \in \mathcal{R}(N_U) : \varphi(m) \leq 1\} = \mathcal{R}(N_U) \setminus V$*. (ii) The projection of* $\mathcal{R}(N_S)$ *onto the plant places equals* Σ*. (iii) Every transition of* N_U *between two markings of* Σ *is enabled in* N_S *at the corresponding marking. Hence* N_S *allows every marking and every transition that any deadlock-free nonblocking supervisor could allow.*

Proof. Let $S = \mathcal{R}(N_U) \setminus V = \{\varphi \leq 1\}$. $\Sigma \subseteq S$*:* markings of V cannot reach m_{f} (Proposition 5), so violate (b). S *satisfies (a):* completions preserve φ. *Lifting:* for $m \in S$ let $\hat{m} = (m,\ C = 1 - \varphi(m))$. Take a plant path $m_0 \to \cdots \to m$. Since V is closed, the path stays in S. Each step $t : m_i \to m_{i+1}$ is enabled in N_S at $\hat{m}_i$: for $t \notin \{\ell_A, \ell_B\}$ the control place adds no precondition; for a load, $m_{i+1} \in S$ gives $\varphi(m_i) = \varphi(m_{i+1}) - 1 = 0$, so $C = 1$. Moreover the firing updates C to $1 - \varphi(m_{i+1})$. Thus $\hat{m} \in \mathcal{R}(N_S)$, and the same argument applied to any plant transition between two markings of S proves (iii). S *satisfies (b):* by Theorem 6(iii), $\hat{m}$ can reach m_{f} in N_S; the projected path stays in $\{\varphi \leq 1\} = S$ by (i) of Theorem 6. Hence S satisfies (a) and (b), so $S \subseteq \Sigma$; with the first inclusion $\Sigma = S$, proving (i). For (ii): $\supseteq$ is the lifting, and $\subseteq$ holds because $\varphi \leq 1$ on $\mathcal{R}(N_S)$ and projected runs are plant runs. The last sentence follows from the remark preceding the theorem. □

Remark 9. Theorem 8 is an instance of the known result that a monitor place enforcing an admissible GMEC is maximally permissive for that constraint [15, 30]. What is specific here is that the constraint $\varphi \leq 1$ is not merely one safe specification: it *coincides* with the supremal nonblocking behaviour, so enforcing it is both necessary and sufficient for deadlock-free completion. (Maximally permissive deadlock avoidance is NP-hard in general [23]; this cell is small and structured enough for a closed-form answer.)

Proposition 10 (Transparency)**.** *Fix delays, arrival times, and the dispatch priority order. If the unrestricted cell* N_U *completes the batch, then the supervised cell* N_S *produces the identical firing log (hence identical makespan, waiting and utilisation). Equivalently, the supervisor changes a run only if unrestricted dispatch would have deadlocked.*

Proof. By induction over the dispatch steps. At a dispatch step the unrestricted dispatcher fires the first enabled immediate transition t. If t were a load leading to $\varphi = 2$, the run would enter V and could not complete (Proposition 5), contradicting the assumption; so t does not enter V. In N_S, t is enabled at the corresponding marking (as in the proof of Theorem 8), and no transition earlier in the priority order is enabled, because enabling in N_S implies enabling in N_U. Timed transitions do not involve C. Hence both runs fire the same transitions at the same times. □

Proposition 11 (State-space sizes)**.** *For* $x = n_A$*,* $y = n_B$*:*

$$|\mathcal{R}(N_U)| = 21xy + 5x + 5y - 7,$$
$$|\mathcal{R}(N_S)| = 17xy + 5x + 5y - 7,$$
$$|\mathcal{R}(N_O)| = 9xy + 5x + 5y + 1.$$

N_U *has* xy *terminal deadlocks and* $4xy$ *doomed markings;* N_S *and* N_O *have none. The conservative policy* O *gives up* $|\mathcal{R}(N_S)| - |\mathcal{R}(N_O)| = 8(xy - 1)$ *safe markings. The number of*

reachable markings in which both machines are processing simultaneously *is* $3xy-2$ *for U,* $2xy-2$ *for S, and* 0 *for O.*

Derivation. By (I1)–(I4) a marking is determined by (1) the *machine configuration* (c_1, c_2) with $c_1 \in \{\text{free}, A_1^{\text{p}}, A_1^{\text{d}}, B_2^{\text{p}}, B_2^{\text{d}}\}$ and $c_2 \in \{\text{free}, B_1^{\text{p}}, B_1^{\text{d}}, A_2^{\text{p}}, A_2^{\text{d}}\}$ and (2) the split of the parts not on machines between queue and output, which can be chosen freely: $(x-k_A+1)(y-k_B+1)$ choices if k_A (k_B) parts of A (B) are on machines. Of the 25 configurations, the 4 with $c_1 \in \{B_2^{\text{p}}, B_2^{\text{d}}\}$ and $c_2 \in \{A_2^{\text{p}}, A_2^{\text{d}}\}$ are unreachable: at the first marking containing both a stage-2 B part (on M_1) and a stage-2 A part (on M_2), the last firing was x_A or x_B. But x_A requires $A_1^{\text{d}} = 1$, an A part on M_1, while a stage-2 B part already holds M_1, contradicting (I1); symmetrically x_B contradicts (I2). The 21 remaining configurations group as follows.

(k_A, k_B)	configurations	splits	note
$(0,0)$	1	$(x+1)(y+1)$	cell empty
$(1,0)$	4	$x(y+1)$	one A
$(0,1)$	4	$(x+1)y$	one B
$(2,0)$	4	$(x-1)(y+1)$	two A
$(0,2)$	4	$(x+1)(y-1)$	two B
$(1,1)$	4	xy	stage-1 A and stage-1 B (region V)

Summing gives $21xy+5x+5y-7$. Removing V ($4xy$ markings) gives $|\mathcal{R}(N_S)|$ by Theorem 8. Under O at most one part is in the cell: configurations $(0,0)$, $(1,0)$, $(0,1)$ only, giving $9xy+5x+5y+1$. Both machines process in configurations $(A_1^{\text{p}}, B_1^{\text{p}}) \in V$ (xy splits), $(A_1^{\text{p}}, A_2^{\text{p}})$ ($(x-1)(y+1)$) and $(B_2^{\text{p}}, B_1^{\text{p}})$ ($(x+1)(y-1)$), totalling $3xy-2$; without V, $2xy-2$. □

The derivation proves everything except that each of the 21 admitted configurations is reachable with every split; this is constructive but tedious, and is confirmed by exhaustive enumeration for all $1 \le n_A, n_B \le 6$ (36 batch shapes; Section 8).

Proposition 12 (Closed-form makespans)**.** *Let all parts be available at* $t=0$ *and processing times be deterministic. Under the dispatch rules of Section 3.4 (either load priority),*

$$T_S = P_A + P_B, \tag{1}$$

$$P_A = \alpha_1 + (n_A-1)\max(\alpha_1,\alpha_2) + \alpha_2, \qquad P_B = \beta_1 + (n_B-1)\max(\beta_1,\beta_2) + \beta_2, \tag{2}$$

$$T_O = n_A(\alpha_1+\alpha_2) + n_B(\beta_1+\beta_2), \tag{3}$$

$$T_O - T_S = (n_A-1)\min(\alpha_1,\alpha_2) + (n_B-1)\min(\beta_1,\beta_2). \tag{4}$$

The relative saving is below $\max\{(n_A-1)/(2n_A), (n_B-1)/(2n_B)\} < \frac{1}{2}$*; for* $n_A = n_B = 2$ *it is at most* 25%*.*

Proof. Policy O. Each part runs alone: its transfer is immediate because the second machine is free, so it occupies the cell for exactly $\alpha_1+\alpha_2$ (or $\beta_1+\beta_2$), and the next part loads at the same instant it unloads. *Policy S.* Say A loads first. After the transfer of A_i the permission token returns, and the dispatcher immediately loads A_{i+1} (if any): ℓ_B is disabled because A_i holds M_2, so ℓ_A is the only enabled load. Thus C is unavailable from the first load until the last A transfers, and the first B load needs M_2, which is freed only when the last A unloads. The first B load therefore occurs exactly at the end of the A phase, and by symmetry the B phase has the same structure, so $T_S = P_A + P_B$. Within the A phase, let τ_i be the transfer time of A_i: $\tau_1 = \alpha_1$ and A_{i+1} starts on M_1 at τ_i, finishes at $\tau_i+\alpha_1$, and needs M_2, which A_i frees at $\tau_i+\alpha_2$; hence $\tau_{i+1} = \tau_i + \max(\alpha_1,\alpha_2)$ and the last unload is at $\tau_{n_A}+\alpha_2 = P_A$. The difference follows from $\alpha_1+\alpha_2-\max(\alpha_1,\alpha_2) = \min(\alpha_1,\alpha_2)$. For the bound, $\min(\alpha_1,\alpha_2) \le (\alpha_1+\alpha_2)/2$, so the ratio of sums is at most the larger of the two ratios. □

Table 3: Summary of closed forms ($x = n_A$, $y = n_B$). All are proved in this section except where marked †, which are confirmed computationally (Section 8).

	U (unrestricted)	O (one active)	S (supervisor)
Reachable markings	$21xy{+}5x{+}5y{-}7^\dagger$	$9xy{+}5x{+}5y{+}1^\dagger$	$17xy{+}5x{+}5y{-}7^\dagger$
Terminal deadlocks	xy	0	0
Doomed markings	$4xy$	0	0
Markings with both machines processing	$3xy{-}2^\dagger$	0	$2xy{-}2^\dagger$
Complete run length (firings)	$5N$ or deadlock	$5N$	$5N$
Makespan, batch at $t{=}0$	deadlock	$x(\alpha_1{+}\alpha_2){+}y(\beta_1{+}\beta_2)$	$P_A{+}P_B$

```
place A_queue 2          # place <name> <initial tokens>
place M1_free 1
place CTRL 1
transition A_unload 1 0  # transition <name> <controllable> <delay>
transition A_xfer   1 0  # file order = dispatch priority
transition A_load   1 0
transition A_done_M1 0 4 # uncontrollable, 4-tick delay
pre  A_load A_queue 1    # pre|post <transition> <place> <weight>
pre  A_load M1_free 1
pre  A_load CTRL    1
post A_load A_M1_proc 1
post A_xfer CTRL    1    # the transfer returns the permission
```

Figure 4: Excerpt of the net file of the supervised cell (the complete file has 15 places and 10 transitions).

Observation (optimality; computational). An exhaustive timed search over *all* controller choices on the unrestricted plant, including deliberate idling, returns exactly T_S in every instance tested: all 162 scenarios of the $2A{+}2B$ grid and 42 further instances with up to four parts per product. We conjecture that this holds in general (interleaving the two products can only reduce the pipelining that T_S already exploits) but do not prove it.

6 Implementation and validation

6.1 Architecture

The software has two halves that share a single model file. The Python half (about 1 200 lines) contains the net class, the builder of the three policies, reachability, invariant and reference-supervisor analysis, the timed simulator, an exact-optimum search, the experiments, and the cross-validation harness. The C++17 half (about 450 lines, header-only, no dependencies) loads a net and executes it. Python exports the net in a plain-text format (Figure 4); the C++ engine reads *that file*, so the two implementations execute literally the same model. The order of transitions in the file is the dispatch priority. The engine has three modes: `simulate` (timed run with an event log), `replay` (apply a given sequence of firings and arrivals and print the marking and the enabled set after every step), and `bench` (timing and memory).

6.2 Execution algorithm

Algorithm 1 is one event-time step; it is implemented identically in Python and C++. In spirit it resembles a programmable-controller scan—inputs (arrivals, expired timers) are read, commands (controllable firings) are computed, timers are updated—but we make no claim of

Algorithm 1 One event-time step of the execution engine (current time t, marking m, timers θ).

inject all external arrivals due at t into m
for all timed transitions u in net order with $\theta_u = t$ **do** fire u; clear θ_u
end for
while some immediate transition is enabled **do** fire the *first* enabled immediate transition in net order
end while ▷ dispatch
for all timed transitions u **do**
    if u is enabled and θ_u unset **then** $\theta_u \leftarrow t + \delta(u)$
    else if u is not enabled **then** clear θ_u
    end if
end for
$t \leftarrow \min(\{\theta_u\} \cup \{\text{next arrival time}\})$; **stop** if this set is empty

real-time behaviour. Each step scans the ten transitions, each with at most three input arcs, so its cost is constant for this net; the model occupies 2.5 kB, and with logging off no memory is allocated per step (one marking copy per run).

6.3 Cross-validation protocol

The two implementations are compared on four layers.

1. **Differential testing of timed runs.** 300 random scenarios × 3 policies = 900 runs, with processing times drawn from $\{1, \ldots, 9\}$, arrival spreads in $\{0, 3, 10, 30\}$ and random load priority. The event logs must be identical, byte for byte.

2. **Lock-step replay.** Every timed log (900), 300 untimed random walks (100 per policy, run until no transition is enabled, hence including deadlocking runs) and the shortest deadlock trace of the unrestricted net: 1 201 sequences. After every step both implementations print the marking and the set of enabled transitions; the outputs must be identical.

3. **Rejected commands.** 14 illegal firings (e.g. a transfer before anything is loaded, a load while the machine is held, an unknown transition, and for policies O and S a second load while the permission token is taken) must be rejected by both with the same diagnostic.

4. **Non-vacuity.** (a) Feeding the C++ engine a *different* net makes the harness report a mismatch. (b) A mutated supervisor that returns the permission at the stage-1 completion events instead of at the transfers is found to deadlock by the exhaustive checker, as predicted by Section 4. (c) Hand-computed cases: with all processing times equal to 1, the unrestricted cell deadlocks at $t = 1$, policy O finishes at $t = 8$ and policy S at $t = 6$.

This protocol detects any divergence between the implementations *given the same net file*. It cannot detect an error in the shared semantics; those are covered by the independent checks (4c), by the closed forms of Section 5, and by Lemma 3, which fixes the number of firings of a complete run at $5N$ (20 and 1000 firings for the two benchmark batches).

6.4 Tests and reproducibility

A suite of 193 automated tests covers net semantics, invariants, deadlock counts, closed-form makespans for 81 timing combinations under both priorities, the flawed-supervisor mutation,

and the cross-checks. A separate script machine-checks every closed-form claim of Section 5 (Section 8). One command (`make reproduce`) builds the engine, runs the tests, the verification, both experiments, the closed-form checks, the cross-validation and the benchmark, draws the figures, writes the results report, and ends with a PASS/FAIL summary. All randomness is seeded and time is integral, so every output except the benchmark is deterministic. The deterministic result files (the two experiment tables, the verification and scaling data, the experiment summary, the cross-validation summary, and the exported net and matrices) have identical SHA-256 digests on two environments: (A) AMD Ryzen 7 6800H, Ubuntu 22.04 (Linux 6.8.0-138, glibc 2.35), Python 3.10.12, g++ 11.4.0; and (B) an Intel Xeon at 2.8 GHz, Ubuntu 24.04, Python 3.12.3, g++ 13.3.0.

7 Experimental design

Exhaustive verification. For the $2A{+}2B$ batch we enumerate $\mathcal{R}(N)$ of each policy and report markings, transitions, terminal deadlocks, doomed markings, invariants, and the number of states with both machines processing. We compare each supervisor with the maximally permissive reference Σ at the level of both markings and transitions. For the closed forms, we repeat the enumeration for all $1 \le n_A, n_B \le 6$ and check N_U, N_O, N_S against Proposition 11.

Experiment 1: deterministic grid. Batch $2A{+}2B$ available at $t = 0$; all $3^4 = 81$ combinations $(\alpha_1, \alpha_2, \beta_1, \beta_2) \in \{2, 4, 8\}^4$ and both load priorities, giving 162 scenarios per policy. Metrics: makespan; machine utilisation (processing time divided by makespan); mean waiting time (arrival to load); mean flow time (arrival to unload); and blocking (total ticks machines hold finished parts). The exact optimum comes from a memoised search over states (marking, remaining timer values) whose actions are "fire an enabled controllable transition now" and "advance time to the next machine completion"; it ranges over every controller, including idling.

Experiment 2: Monte Carlo with staggered arrivals. Each of the four parts arrives at an integer time drawn uniformly from $[0, s]$, with spread $s \in \{0, 2, 4, 8, 16, 32, 64, 128\}$ ticks, for four time classes: balanced $(4, 4, 4, 4)$; M_2-bottleneck $(2, 8, 8, 2)$; long first stages $(8, 2, 8, 2)$; and a class drawn per scenario from $\{2, 4, 8\}^4$. Load priority is random. Each cell has 500 scenarios ($4 \times 8 \times 500 = 16\,000$ in total), and all three policies see *identical* scenarios (common random numbers [22]). Deadlock probabilities carry Wilson 95% intervals [39]; the mean paired makespan saving of S over O carries a percentile bootstrap [10] 95% interval (2 000 resamples).

Engine benchmark. The supervised net with balanced times is executed for the demo batch $2A{+}2B$ and a long batch $100A{+}100B$. After 50 warm-up runs we report the mean over 100 000 (respectively 2 000) runs; the per-step latency distribution is taken from up to 20 000 runs with a monotonic clock (two clock reads per step are included in each sample); peak resident memory is the high-water mark of the engine process read from `/proc/self/status`, taken before the latency buffer is allocated. The Python reference is timed on the same scenario (2 000 and 20 runs) and its heap peak is measured with `tracemalloc`. All benchmark numbers are from environment (A) and are machine-dependent.

Table 4: Exhaustive verification of the three policies, batch $2A{+}2B$ (U: unrestricted, O: one active part, S: control-place supervisor). "Edges" counts transitions of the reachability graph; "both processing" counts reachable markings in which both machines process at once. All P-invariants hold on every reachable marking of every policy.

Policy	Places	Markings	Edges	Terminal deadlocks	Doomed markings	Both processing	Max parts in cell
U	14	97	140	4	16	10	2
O	15	57	60	0	0	0	1
S	15	81	108	0	0	6	2
Reference Σ (Theorem 8)	—	81	108	0	0	—	—

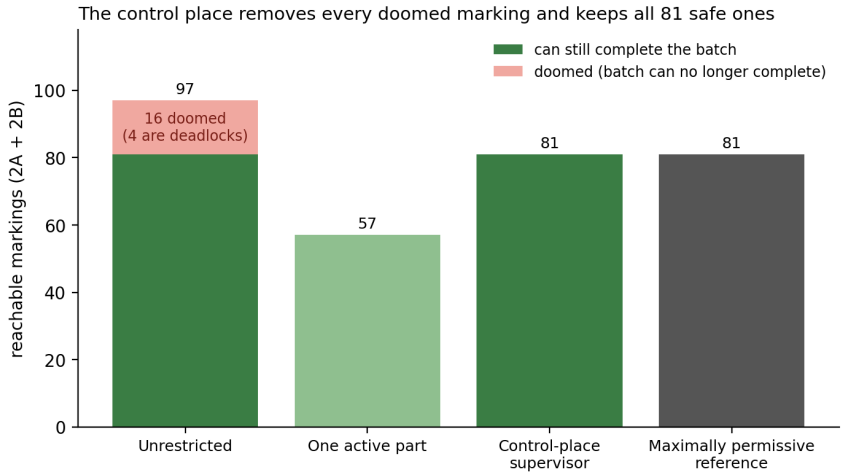


Figure 5: Reachable markings for the $2A{+}2B$ batch. The control place removes exactly the 16 doomed markings of the unrestricted plant and equals the maximally permissive reference (81 markings).

8 Results

8.1 Exhaustive verification

Table 4 and Figure 5 give the central picture. Unrestricted dispatch reaches 97 markings, of which 16 are doomed and 4 are terminal deadlocks; all 16 vanish under the control place, which retains the remaining 81 markings and all 108 transitions between them—exactly the maximally permissive reference Σ at the level of markings *and* transitions. The conservative policy O keeps only 57 markings (it gives up 24 safe markings and 48 safe transitions) and admits no state in which both machines process simultaneously, whereas S retains 6 of the 10 such states of the plant (the other 4 are doomed). All invariants hold on every reachable marking of every policy, and all machine and control places are 1-bounded.

8.2 Closed forms

Table 5 compares enumeration with Proposition 11 for balanced batches. The complete machine check covers all 36 batch shapes with up to six parts per product: the formulas for markings, terminal deadlocks (xy), doomed markings ($4xy$) and states with both machines processing hold in every case (0 mismatches). For general batches, T_S, T_O and the saving of

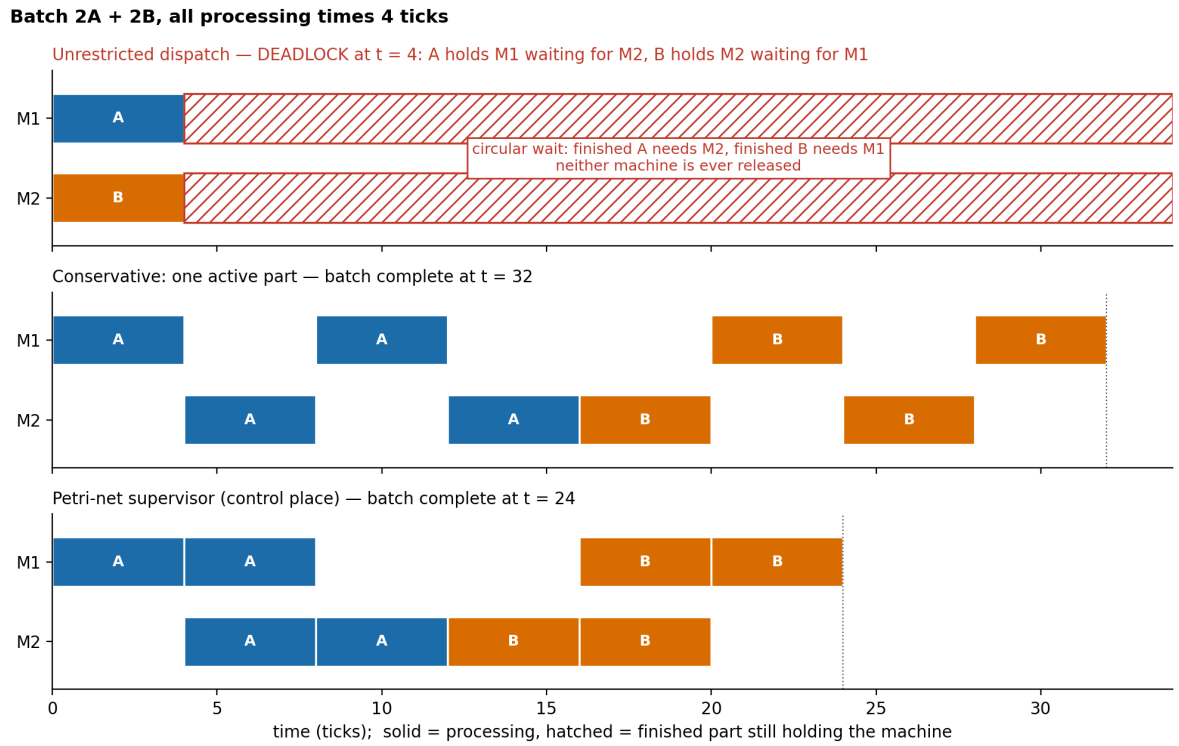


Figure 6: Gantt charts for the $2A{+}2B$ batch with all processing times equal to 4 ticks. Top: unrestricted dispatch loads A on M_1 and B on M_2 at $t = 0$ and deadlocks at $t = 4$ (hatched: finished parts holding the machines). Middle: one active part, makespan 32. Bottom: control place, makespan 24; parts of the same product pipeline through the machines.

Proposition 12 matched 800 simulations (shapes up to $4A{+}4B$, random times in $\{1, \ldots, 9\}$, both priorities) with 0 mismatches, and the exact optimum equalled T_S in all 42 larger instances. The benchmark makespan for $100A{+}100B$ with all times 4 is 808 ticks, which agrees with $P_A + P_B = 404 + 404$.

8.3 Experiment 1: deterministic timing grid

Unrestricted dispatch deadlocked in all 162 scenarios, because greedy dispatch loads an A on M_1 and a B on M_2 at $t = 0$. Both controlled policies completed every scenario. The supervisor shortened the makespan relative to policy O by 17.8% on average (range 10.0%–25.0%) and was strictly faster in all 162 scenarios; in every scenario the saving equalled $\min(\alpha_1, \alpha_2) + \min(\beta_1, \beta_2)$ ticks (Proposition 12, Figure 7b), and the supervisor attained the exact optimum in 162 of 162 scenarios (maximum gap 0.00%). It raised mean machine utilisation from 50% to 61% and reduced mean waiting time from 14.0 to 10.0 ticks, at the cost of 2.7 ticks of blocking per scenario (machines holding finished parts), which the conservative policy never incurs.

8.4 Experiment 2: staggered arrivals

Table 7 and Figure 8 summarise the Monte Carlo study. (i) Both controlled policies had **0 deadlocks in all 32 000 controlled runs**. (ii) Unrestricted dispatch deadlocked with probability 1.00 when all parts arrive together and remained dangerous even when the cell was lightly loaded: with arrivals spread over 128 ticks (against 4-tick operations) the deadlock probability was still 0.20 (balanced) to 0.42 (long first stages). (iii) The advantage of the

Table 5: Scaling with the batch size (balanced batches nA + nB): enumeration versus the closed forms of Proposition 11. "Gives up" is the number of safe markings the conservative policy O forgoes ($= 8(n^2 - 1)$). The last column marks agreement of all closed forms and of $S = \Sigma$ with enumeration.

Batch	U markings	U deadlocks	O markings	S markings	Reference Σ	O gives up	Formulas
1A+1B	24	1	20	20	20	0	✓
2A+2B	97	4	57	81	81	24	✓
3A+3B	212	9	112	176	176	64	✓
4A+4B	369	16	185	305	305	120	✓

Table 6: Experiment 1: means over the 162 scenarios per policy (81 time combinations × 2 load priorities; batch $2A{+}2B$ at $t = 0$). Utilisation is processing time divided by makespan; blocking is the total number of ticks in which machines hold finished parts. The exact optimum is the minimum makespan over all controllers.

Policy	Deadlocked runs	Makespan (ticks)	Utilisation M_1 / M_2	Waiting / flow time	Blocking (ticks)
U (unrestricted)	162 / 162	—	—	—	—
O (one active part)	0 / 162	37.3	50% / 50%	14.0 / 23.3	0.0
S (supervisor)	0 / 162	30.7	61% / 61%	10.0 / 20.0	2.7
Exact optimum	—	30.7	—	—	—

supervisor over policy O is largest when the cell is loaded (25% for the balanced class at spread 0) and vanishes as arrivals spread out (0.4% at spread 128), because concurrency only matters when parts compete. (iv) In the 4 140 of 16 000 scenarios in which unrestricted dispatch happened to survive, the supervisor's makespan was identical to the unrestricted makespan in every one (mean difference exactly zero), as Proposition 10 predicts; a separate check on 3 000 fresh random scenarios (batches up to $4A{+}4B$) found identical firing logs in all 743 scenarios in which the unrestricted cell completed, and 0 supervisor deadlocks.

8.5 Cross-validation

Table 8 reports the cross-validation outcome: no mismatch in any layer, so the Python reference and the C++ engine agree on every event sequence tested, and both reject exactly the same illegal commands.

8.6 Engine performance

Table 9 gives the resource measurements on environment (A). The engine executes one transition firing in 9–24 ns on average (the demo batch includes fixed per-run overhead), a complete $2A{+}2B$ batch in 0.49 μs, and the $100A{+}100B$ batch (1 000 firings) in 9.3 μs. The Python reference is 79 and 149 times slower, respectively. Peak resident memory is 3.4 MB, dominated by the C++ runtime and shared libraries; the model itself occupies 2.5 kB. The step-latency maximum is operating-system scheduling jitter on a non-real-time system and must *not* be read as a worst-case execution time.

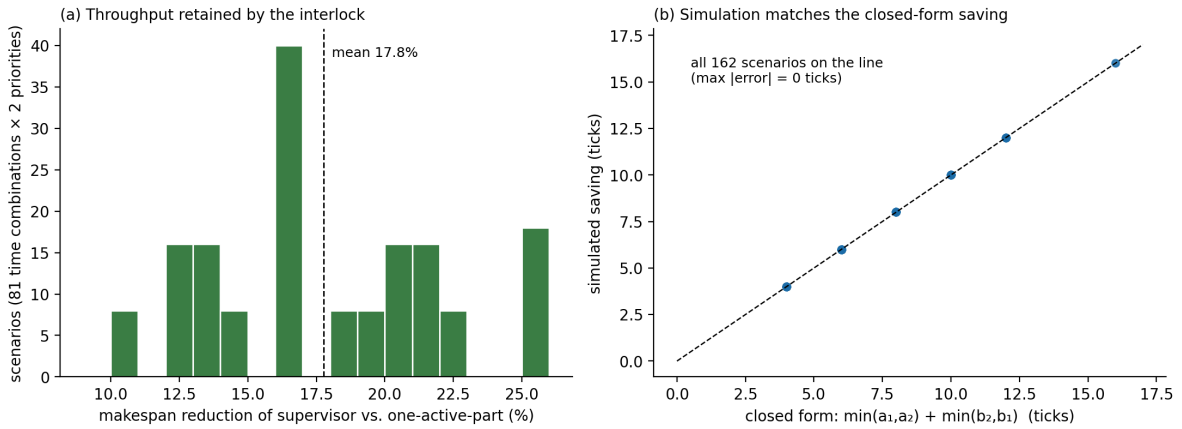


Figure 7: Experiment 1. (a) Distribution of the makespan reduction of S relative to O over 162 scenarios. (b) Simulated saving versus the closed form $\min(\alpha_1, \alpha_2) + \min(\beta_1, \beta_2)$; all scenarios lie on the diagonal.

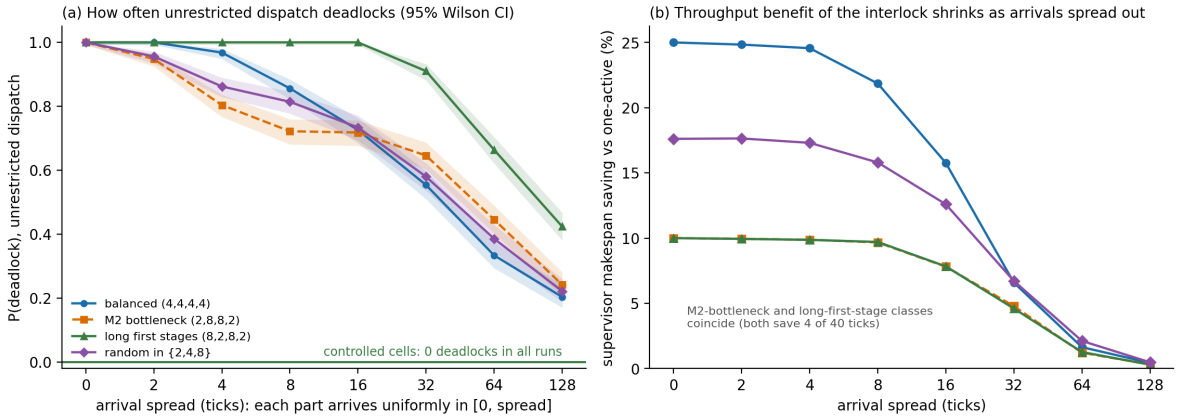


Figure 8: Experiment 2 (500 scenarios per point). (a) Probability that unrestricted dispatch deadlocks, with Wilson 95% bands; controlled cells never deadlocked. (b) Mean makespan saving of S relative to O (percent). The M_2-bottleneck and long-first-stage classes coincide because both save 4 of 40 ticks.

9 Discussion

9.1 Answering the research question

Logically, the control place retains all the concurrency that any deadlock-free supervisor could: it is maximally permissive in both markings and transitions (Theorem 8) for every batch size, and it is transparent—it never changes a run that unrestricted dispatch would have completed (Proposition 10, confirmed on 4 140 scenarios of Experiment 2 and on 743 fresh ones). *In time,* compared with the conservative one-part-at-a-time policy it shortens the makespan by 10–25% on the deterministic grid, with the closed-form saving $(n_A - 1)\min(\alpha_1, \alpha_2) + (n_B - 1)\min(\beta_1, \beta_2)$; it attains the exact optimum in every instance we tested; and its advantage disappears when arrivals are sparse, which is expected because concurrency only matters under contention. *Computationally,* executing the supervisor costs tens of nanoseconds per firing and a few megabytes, so for a cell of this size the control logic is not a resource concern.

Table 7: Experiment 2 (500 scenarios per row; common random numbers). **Top:** balanced time class $(4, 4, 4, 4)$; deadlock probability of unrestricted dispatch with Wilson 95% interval, mean makespans of S and O, and the mean paired saving of S relative to O with bootstrap 95% interval. Controlled policies had no deadlock in any run. **Bottom:** other time classes, "deadlock probability of U / saving of S over O in %".

Spread (ticks)	P(deadlock), U	Makespan S	Makespan O	Saving S vs O (ticks)	Saving (%)
0	1.00 [0.99, 1.00]	24.0	32.0	8.0 [8.0, 8.0]	25.0
2	1.00 [0.99, 1.00]	24.2	32.2	8.0 [8.0, 8.0]	24.8
4	0.97 [0.95, 0.98]	24.6	32.6	8.0 [8.0, 8.0]	24.6
8	0.86 [0.82, 0.88]	26.1	33.3	7.3 [7.2, 7.4]	21.9
16	0.73 [0.69, 0.76]	29.4	34.9	5.5 [5.3, 5.7]	15.8
32	0.55 [0.51, 0.60]	38.0	40.8	2.7 [2.5, 3.0]	6.6
64	0.33 [0.29, 0.38]	61.8	62.7	1.0 [0.8, 1.1]	1.6
128	0.20 [0.17, 0.24]	110.6	111.0	0.4 [0.3, 0.5]	0.4

Time class $(\alpha_1, \alpha_2, \beta_1, \beta_2)$	spread 8	spread 32	spread 128
M2 bottleneck (2,8,8,2)	0.72 / 9.7%	0.65 / 4.8%	0.24 / 0.3%
long first stages (8,2,8,2)	1.00 / 9.7%	0.91 / 4.6%	0.42 / 0.3%
random in {2,4,8}	0.81 / 15.8%	0.58 / 6.7%	0.22 / 0.5%

Table 8: Cross-validation of the Python reference against the C++ engine.

Layer	Cases	Mismatches
1. Timed runs, event logs compared byte for byte	900	0
2. Lock-step replay (23 465 steps; incl. 1 deadlock trace)	1 201	0
3. Illegal commands rejected identically	14	0
4. Automated tests (net semantics, closed forms, mutations)	193	0 failures

9.2 Lessons

1. **Where the permission returns is the design decision.** Returning the token at the transfer—when the part actually leaves the machine it holds—is correct; returning it at the stage-1 completion admits the circular wait, and the exhaustive checker finds the resulting deadlock immediately.

2. **Doomed precedes deadlocked.** In the $2A{+}2B$ cell, 12 doomed markings are not yet terminal. A monitor that merely waits for the cell to stall reacts after the point of no return; prevention has to act on the *commands*.

3. **Transparency is a deployment argument.** Because the supervisor intervenes only in runs that would deadlock, it can be introduced without altering behaviour in normal operation—within the assumptions of Section 3.

4. **Token abstraction keeps the problem small.** Treating parts of one product as interchangeable tokens makes the state space grow quadratically with the batch (Proposition 11); models that track individual part identities would be larger, which we did not study.

Table 9: Engine resource measurements on AMD Ryzen 7 6800H with Radeon Graphics (x86_64), Ubuntu 22.04 (Linux 6.8.0-138), g++ 11.4.0 with `-O2 -std=c++17`, Python 3.10.12. Supervised net, all processing times 4 ticks. The step-latency maximum is OS jitter, not a worst-case bound.

Metric	$2A{+}2B$	$100A{+}100B$
Makespan (ticks)	24	808
Transition firings per run ($= 5N$)	20	1000
C++ time per run (μs)	0.49	9.34
C++ time per firing (ns)	24.4	9.3
C++ step latency mean / p50 / p99 (ns)	51 / 50 / 61	60 / 60 / 71
C++ step latency maximum (ns)	5 260	5 581
Python time per run (μs)	38.5	1 388.5
Speed-up of C++ over Python	$\times$79	$\times$149
C++ peak resident memory (MB)	3.4	3.4
Model data (kB)	2.5	2.5
Python heap peak (kB)	1.0	1.4

9.3 Limitations and threats to validity

Model scope. Two machines, two opposite routes, reliable machines, atomic transfers, no buffers, deterministic times, known arrivals. Machine failures can create blocking that the supervisor was not designed for; the guarantees of Section 5 assume (A1)–(A6). Larger layouts have more siphons and typically need several monitors, and optimal avoidance is NP-hard in general [23].

Parameter coverage. Experiment 1 uses times in $\{2, 4, 8\}$ and Experiment 2 a uniform arrival model. The deadlock *rates* depend on these choices and are illustrative; the existence of the deadlock and its prevention are structural and do not. Confidence intervals reflect Monte Carlo error, not model uncertainty.

Implementation validity. The cross-validation shares the net file, so a conceptual error in the model would not be exposed by it; the independent checks of Section 6.3, the closed forms and Lemma 3 mitigate this. The analysis code and the engine have not been compared with an established tool such as TINA [3], and the engine is not formally verified. Parts of the software and of this manuscript were produced with the assistance of a language model (see Acknowledgments); all results were regenerated and checked by the author's own runs.

Performance claims. Benchmarks come from one laptop running a non-real-time operating system and execute a *simulated* plant; they include no fieldbus or I/O latency and give no worst-case execution time. Nothing was run on a PLC, an embedded board, or a physical machine.

Novelty. The monitor-place technique is established [15, 30, 41]; this report does not propose a new algorithm.

9.4 Future work

(1) Compare the analysis against TINA (marking graphs, semiflows) [3]. (2) Add machine failures and supervisors robust to them. (3) Extend to buffers, more machines and routes, using siphon-based control and GMEC synthesis [11, 24]. (4) Replace deterministic delays by

firing intervals [2, 29] to cover processing-time uncertainty. (5) Run the engine on an embedded Linux board and in hardware-in-the-loop with a PLC, and compare interpretation with code generation to IEC 61131-3 [18, 21]. (6) Formally verify the engine's step function. (7) Study the interface between discrete supervision and continuous constraint-enforcing control layers [9, 13].

10 Conclusion

For the smallest blocking cell with opposite routings, we showed exactly how and when deadlock arises—$n_A n_B$ terminal markings inside an absorbing region of $4n_A n_B$ doomed markings, reachable by a four-firing sequence—and that a single monitor place enforcing $\varphi \leq 1$ is deadlock-free, nonblocking, maximally permissive and transparent for every batch size. Closed forms give the state-space sizes and the makespans, the supervisor shortens the makespan of the conservative policy by up to 25% and reaches the exact optimum in all tested instances, and a C++ execution engine agrees event-for-event with an independent Python reference while using tens of nanoseconds per firing. The result is a small, fully reproducible case study in which the model, the proofs, the experiments and the executable controller can all be checked against one another. Its limits—two machines, reliable machines, simulated signals—are explicit, and extending it to failures, buffers, larger layouts and real hardware is the natural next step.

Code and data availability

All code, exported nets, result files and figures are available at `https://github.com/mohamedghoneim281/deadlock-prevention` (archived at Zenodo, DOI: 10.5281/zenodo.23485309). Running `bash scripts/setup_ubuntu.sh` (or `make reproduce`) regenerates every number and figure in this report and prints a PASS/FAIL summary of the headline claims.

References

[1] Zbigniew A. Banaszak and Bruce H. Krogh. Deadlock avoidance in flexible manufacturing systems with concurrently competing process flows. *IEEE Transactions on Robotics and Automation*, 6(6):724–734, 1990.

[2] Bernard Berthomieu and Michel Diaz. Modeling and verification of time dependent systems using time Petri nets. *IEEE Transactions on Software Engineering*, 17(3):259–273, 1991.

[3] Bernard Berthomieu, Pierre-Olivier Ribet, and François Vernadat. The tool TINA – construction of abstract state spaces for Petri nets and time Petri nets. *International Journal of Production Research*, 42(14):2741–2756, 2004.

[4] Christos G. Cassandras and Stéphane Lafortune. *Introduction to Discrete Event Systems*. Springer, 2nd edition, 2008.

[5] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.

[6] E. G. Coffman, M. Elphick, and A. Shoshani. System deadlocks. *ACM Computing Surveys*, 3(2):67–78, 1971.

[7] René David and Hassane Alla. *Discrete, Continuous, and Hybrid Petri Nets*. Springer, 2nd edition, 2010.

[8] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[9] Stefano Di Cairano, Abraham Goldsmith, Uroš Kalabić, and Scott A. Bortoff. Cascaded reference governor–MPC for motion control of two-stage manufacturing machines. *IEEE Transactions on Control Systems Technology*, 2018. doi: 10.1109/TCST.2018.2847627.

[10] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[11] Joaquín Ezpeleta, José Manuel Colom, and Javier Martínez. A Petri net based deadlock prevention policy for flexible manufacturing systems. *IEEE Transactions on Robotics and Automation*, 11(2):173–184, 1995.

[12] Maria Pia Fanti and MengChu Zhou. Deadlock control methods in automated manufacturing systems. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 34(1):5–22, 2004.

[13] Emanuele Garone, Stefano Di Cairano, and Ilya Kolmanovsky. Reference and command governors for systems with constraints: A survey on theory and applications. *Automatica*, 75:306–328, 2017.

[14] Alessandro Giua and Frank DiCesare. Blocking and controllability of Petri nets in supervisory control. *IEEE Transactions on Automatic Control*, 39(4):818–823, 1994.

[15] Alessandro Giua, Frank DiCesare, and Manuel Silva. Generalized mutual exclusion constraints on nets with uncontrollable transitions. In *Proceedings of the IEEE International Conference on Systems, Man and Cybernetics*, pages 974–979, Chicago, IL, 1992.

[16] Nicholas G. Hall and Chelliah Sriskandarajah. A survey of machine scheduling problems with blocking and no-wait in process. *Operations Research*, 44(3):510–525, 1996.

[17] Lawrence E. Holloway, Bruce H. Krogh, and Alessandro Giua. A survey of Petri net methods for controlled discrete event systems. *Discrete Event Dynamic Systems*, 7(2): 151–190, 1997.

[18] International Electrotechnical Commission. *IEC 61131-3: Programmable Controllers — Part 3: Programming Languages*, 3.0 edition, 2013.

[19] Marian V. Iordache and Panos J. Antsaklis. *Supervisory Control of Concurrent Systems: A Petri Net Structural Approach*. Birkhäuser, 2006.

[20] Kurt Jensen, Lars M. Kristensen, and Lisa Wells. Coloured Petri nets and CPN tools for modelling and validation of concurrent systems. *International Journal on Software Tools for Technology Transfer*, 9(3–4):213–254, 2007.

[21] Hisham Kaid, Abdulrahman Al-Ahmari, Abdelmaged M. El-Tamimi, Emad A. Nasr, and Zhiwu Li. Design and implementation of deadlock control for automated manufacturing systems. *South African Journal of Industrial Engineering*, 30(1):1–23, 2019.

[22] Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill Education, 5th edition, 2015.

[23] Mark A. Lawley and Spyros A. Reveliotis. Deadlock avoidance for sequential resource allocation systems: Hard and easy cases. *International Journal of Flexible Manufacturing Systems*, 13, 2001.

[24] Zhiwu Li and MengChu Zhou. *Deadlock Resolution in Automated Manufacturing Systems: A Novel Petri Net Approach*. Advances in Industrial Control. Springer, 2009.

[25] Zhiwu Li, MengChu Zhou, and Naiqi Wu. A survey and comparison of Petri net-based deadlock prevention policies for flexible manufacturing systems. *IEEE Transactions on Systems, Man, and Cybernetics – Part C: Applications and Reviews*, 38(2):173–188, 2008.

[26] Zhiwu Li, Naiqi Wu, and MengChu Zhou. Deadlock control of automated manufacturing systems based on Petri nets—a literature review. *IEEE Transactions on Systems, Man, and Cybernetics – Part C: Applications and Reviews*, 42(4):437–462, 2012.

[27] J. Martínez and M. Silva. A simple and fast algorithm to obtain all invariants of a generalised Petri net. In *Application and Theory of Petri Nets*, volume 52 of *Informatik-Fachberichte*, pages 301–310. Springer, 1982.

[28] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10 (1):100–107, 1998.

[29] Philip M. Merlin and David J. Farber. Recoverability of communication protocols—implications of a theoretical study. *IEEE Transactions on Communications*, 24(9):1036–1043, 1976.

[30] John O. Moody and Panos J. Antsaklis. *Supervisory Control of Discrete Event Systems Using Petri Nets*. Kluwer Academic Publishers, 1998.

[31] Tadao Murata. Petri nets: Properties, analysis and applications. *Proceedings of the IEEE*, 77(4):541–580, 1989.

[32] Peter J. G. Ramadge and W. Murray Wonham. The control of discrete event systems. *Proceedings of the IEEE*, 77(1):81–98, 1989.

[33] Chander Ramchandani. *Analysis of Asynchronous Concurrent Systems by Timed Petri Nets*. Project mac technical report tr-120, Massachusetts Institute of Technology, 1974.

[34] Wolfgang Reisig. *Understanding Petri Nets: Modeling Techniques, Analysis Methods, Case Studies*. Springer, 2013.

[35] Spyros A. Reveliotis. *Real-Time Management of Resource Allocation Systems: A Discrete Event Systems Approach*. Springer, 2005.

[36] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013.

[37] N. Viswanadham, Y. Narahari, and T. L. Johnson. Deadlock prevention and deadlock avoidance in flexible manufacturing systems using Petri net models. *IEEE Transactions on Robotics and Automation*, 6(6):713–723, 1990.

[38] Valeriy Vyatkin. IEC 61499 as enabler of distributed and intelligent automation: State-of-the-art review. *IEEE Transactions on Industrial Informatics*, 7(4):768–781, 2011.

[39] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.

[40] W. Murray Wonham and Kai Cai. *Supervisory Control of Discrete-Event Systems*. Springer, 2019.

[41] Katerina Yamalidou, John O. Moody, Michael D. Lemmon, and Panos J. Antsaklis. Feedback control of Petri nets based on place invariants. *Automatica*, 32(1):15–28, 1996.

A Notation and software names

Table 10: Symbols used in the paper and identifiers in the net file.

Symbol	Code name	Meaning
$Q_A,\ Q_B$	`A_queue`, `B_queue`	queued parts
$F_1,\ F_2$	`M1_free`, `M2_free`	machine availability tokens
$A_1^{\mathrm{p}},\ A_1^{\mathrm{d}}$	`A_M1_proc`, `A_M1_done`	A in stage 1 on M_1: processing, finished (holds M_1)
$A_2^{\mathrm{p}},\ A_2^{\mathrm{d}}$	`A_M2_proc`, `A_M2_done`	A in stage 2 on M_2
$B_1^{\mathrm{p}},\ B_1^{\mathrm{d}}$	`B_M2_proc`, `B_M2_done`	B in stage 1 on M_2
$B_2^{\mathrm{p}},\ B_2^{\mathrm{d}}$	`B_M1_proc`, `B_M1_done`	B in stage 2 on M_1
$O_A,\ O_B$	`A_out`, `B_out`	finished parts
C	`CTRL`	control (monitor) place
φ	—	first-stage occupancy $A_1^{\mathrm{p}}+A_1^{\mathrm{d}}+B_1^{\mathrm{p}}+B_1^{\mathrm{d}}$
α_1, α_2	`a_m1`, `a_m2`	processing time of A on M_1, on M_2
β_1, β_2	`b_m2`, `b_m1`	processing time of B on M_2 (stage 1), on M_1 (stage 2)

B Invariants found by the Farkas-type algorithm

For the supervised net N_S the algorithm returns exactly five minimal invariants (machine M_1, machine M_2, the control place, and the two part-conservation laws):

```
M1_free + A_M1_proc + A_M1_done + B_M1_proc + B_M1_done = 1
M2_free + A_M2_proc + A_M2_done + B_M2_proc + B_M2_done = 1
A_M1_proc + A_M1_done + B_M2_proc + B_M2_done + CTRL = 1
A_queue + A_M1_proc + A_M1_done + A_M2_proc + A_M2_done + A_out = 2
B_queue + B_M2_proc + B_M2_done + B_M1_proc + B_M1_done + B_out = 2
```

(for $n_A = n_B = 2$). The plant N_U has the first, second, fourth and fifth; the conservative net N_O replaces the third by a constraint over all eight machine-holding places.

C Claims and the scripts that check them

Table 11: Where each claim is checked in the repository.

Claim	Script / result file
97/57/81 markings; deadlocks; invariants; Σ	`run_verification.py`, `verification.json`
Closed forms for states, deadlocks, doomed, parallelism; makespans; optimum; transparency	`check_closed_forms.py`, `closed_forms.json`
Experiments 1 and 2	`run_experiments.py`, `exp1_grid.csv`, `exp2_montecarlo.csv`
Python $\leftrightarrow$ C++ agreement	`run_crosscheck.py`, `crosscheck.json`
Engine timing and memory	`run_benchmark.py`, `benchmark.json`
All of the above, final PASS/FAIL	`run_all.py`, `check_results.py`